=== UNIT 2: CASCADING STYLE SHEETS (CSS3), BOX MODEL & MODERN LAYOUTS ===

>> 2.1 CSS3 Syntax, Cascading Rules and Specificity Hierarchy

Cascading Style Sheets (CSS3) dictates the presentation, layout, visual aesthetics, and responsive behavior of HTML documents across display viewports.
* CSS Syntax: selector { property: value; }.
* Insertion Methods: (1) Inline Styles (<h1 style='...'>) - highest specificity, violates separation of concerns. (2) Internal Stylesheet (<style> in <head>). (3) External Stylesheet (<link rel='stylesheet' href='style.css'>) - industry standard, enables browser caching.
* The Cascade and Specificity Hierarchy: When conflicting rules target an element, the browser computes Specificity Scores represented as (Inline, IDs, Classes/Attributes/Pseudo-classes, Elements/Pseudo-elements):
- Inline style: (1, 0, 0, 0) = Score 1000.
- ID selector (#header): (0, 1, 0, 0) = Score 100.
- Class, attribute, pseudo-class (.btn, [type='text'], :hover): (0, 0, 1, 0) = Score 10.
- Element, pseudo-element (p, h1, ::before): (0, 0, 0, 1) = Score 1.
- Universal selector (*), combinators (+, >, ~): (0, 0, 0, 0) = Score 0.
- The !important override trumps all specificity, but should be avoided to prevent unmaintainable code.

>> 2.2 The CSS Box Model and Sizing Paradigms

Every rendered HTML element is treated as a rectangular box comprising four concentric layers:
* 1. Content: The actual text, image, or media content area (width x height).
* 2. Padding: Transparent interior clearance between content and border.
* 3. Border: Structural perimeter line wrapping padding and content.
* 4. Margin: Transparent exterior clearance separating the element from neighboring boxes.
* The Critical box-sizing Property:
- content-box (Default): Total Width = width + padding_left + padding_right + border_left + border_right. Adding padding increases physical element size, frequently breaking grid layouts.
- border-box (Universal Best Practice): Total Width = declared width. Padding and border are absorbed inside the declared width: * { box-sizing: border-box; }.

>> 2.3 Display Modes and Element Positioning

* Display Modes: block (occupies 100% width, starts on new line: div, p, h1), inline (occupies only content width, ignores top/bottom margin/padding: span, a), inline-block (flows inline, but respects width, height, padding, margin), none (removes element from DOM layout).
* Positioning Schemes (position):
- static (Default): Normal document flow.
- relative: Positioned relative to its normal flow position (top, left) without affecting surrounding layout.
- absolute: Removed from document flow; positioned relative to nearest positioned ancestor (non-static).
- fixed: Positioned relative to the browser viewport; remains stationary during page scrolling.
- sticky: Toggles between relative and fixed depending on scroll threshold.

>> 2.4 Flexbox (One-Dimensional Flexible Box Layout)

Flexbox aligns items along a single dimension (either row or column) with dynamic space distribution:
* Flex Container Properties: display: flex; flex-direction: row | column; flex-wrap: nowrap | wrap; justify-content: flex-start | center | space-between | space-around | space-evenly (aligns along main axis); align-items: stretch | center | flex-start | flex-end (aligns along cross axis).
* Flex Item Properties: flex-grow (expansion factor), flex-shrink (shrinkage resistance), flex-basis (initial size), align-self (overrides container alignment). Shorthand: flex: 1 1 auto.

>> 2.5 CSS Grid (Two-Dimensional Layout Architecture)

CSS Grid provides simultaneous multi-row and multi-column control:
* Container Properties: display: grid; grid-template-columns: repeat(3, 1fr) (three equal fractional columns); grid-template-rows; gap: 20px (row and column gutters); grid-template-areas: ’header header’ ’nav main’ ’footer footer’.
* Item Placement: grid-column: 1 / 3; grid-row: 2 / 4; grid-area: header.

[PROGRAM 1: Universal CSS Reset and Typography System]

Problem Context: Construct a foundational CSS stylesheet establishing universal border-box sizing, responsive typography using rem units, and standard CSS custom properties (variables).

CSS Source Code:

```
/* Modern CSS Foundation & Custom Properties */
:root {
--primary-color: #0d6efd;
--secondary-color: #6c757d;
--success-color: #198754;
--dark-bg: #212529;
--light-bg: #f8f9fa;
--font-stack: 'Segoe UI', system-ui, -apple-system, sans-serif;
--spacing-unit: 1rem;
}

/* Universal Box-Sizing Reset */
*, *::before, *::after {
box-sizing: border-box;
margin: 0;
padding: 0;
}

body {
font-family: var(--font-stack);
font-size: 1rem;
line-height: 1.6;
color: var(--dark-bg);
background-color: var(--light-bg);
}

h1, h2, h3 { line-height: 1.2; margin-bottom: 0.75rem; }
h1 { font-size: 2.25rem; color: var(--primary-color); }
p { margin-bottom: 1rem; }
```

Explanation: Uses CSS variables (--primary-color), universal border-box reset, and rem-based responsive typography.

[PROGRAM 2: Responsive Flexbox Navigation Bar]

Problem Context: Create a responsive horizontal navigation bar using Flexbox that aligns the brand logo to the left, links to the right, and centers items vertically.

HTML & CSS Code:

```
<!-- HTML Navigation -->
<nav class='navbar'>
<div class='nav-brand'>LPU Engineering</div>
<ul class='nav-links'>
<li><a href='#'>Home</a></li>
<li><a href='#'>Academics</a></li>
<li><a href='#'>Placements</a></li>
<li><a href='#' class='btn-cta'>Student Portal</a></li>
</ul>
</nav>

/* Flexbox CSS */
.navbar {
display: flex;
justify-content: space-between; /* Brand on left, links on right */
align-items: center;            /* Center vertically on cross-axis */
padding: 1rem 2rem;
background-color: #ffffff;
box-shadow: 0 2px 4px rgba(0,0,0,0.1);
}
.nav-brand { font-size: 1.5rem; font-weight: bold; color: #0d6efd; }
.nav-links { display: flex; list-style: none; gap: 1.5rem; align-items: center; }
.nav-links a { text-decoration: none; color: #333; font-weight: 500; transition: color
0.2s; }
.nav-links a:hover { color: #0d6efd; }
.btn-cta { background-color: #0d6efd; color: #fff !important; padding: 0.5rem 1rem;
border-radius: 4px; }
```

Key Properties: justify-content: space-between separates brand and links; gap: 1.5rem provides clean spacing without margin hacks.

[PROGRAM 3: Responsive Multi-Column Layout using CSS Grid]

Problem Context: Build a responsive dashboard layout consisting of Header, Sidebar, Main Content Area, and Footer using CSS Grid template areas.

CSS Grid Source Code:

```
.dashboard-container {
display: grid;
min-height: 100vh;
grid-template-columns: 240px 1fr;          /* 240px sidebar, remaining space for
```

```
content */
grid-template-rows: 70px 1fr 50px;            /* Header, content, footer */
grid-template-areas:
'header  header'
'sidebar main'
'footer  footer';
gap: 15px;
}

.header-area  { grid-area: header;  background: #0d6efd; color: #fff; padding: 1rem; }
.sidebar-area { grid-area: sidebar; background: #e9ecef; padding: 1rem; }
.main-area    { grid-area: main;    background: #ffffff; padding: 1.5rem; border-radius:
6px; }
.footer-area  { grid-area: footer;  background: #212529; color: #fff; text-align:
center; padding: 0.75rem; }
```

Key Properties: grid-template-areas defines an intuitive 2D ASCII layout diagram, automatically positioning sub-elements.

[PROGRAM 4: Interactive Card Grid with Hover Transitions]

Problem Context: Implement an interactive course card gallery using CSS Grid repeat(auto-fit, minmax()) and smooth cubic-bezier hover elevation animations.

CSS Source Code:

```
.card-grid {
display: grid;
grid-template-columns: repeat(auto-fit, minmax(280px, 1fr)); /* Fully responsive
auto-wrapping */
gap: 1.5rem;
padding: 2rem;
}
.course-card {
background: #fff;
border-radius: 8px;
border: 1px solid #dee2e6;
padding: 1.5rem;
box-shadow: 0 2px 6px rgba(0,0,0,0.06);
transition: transform 0.25s cubic-bezier(0.4, 0, 0.2, 1), box-shadow 0.25s ease;
}
.course-card:hover {
transform: translateY(-6px);                  /* Smooth uplifting card animation */
```

box-shadow: 0 12px 20px rgba(0,0,0,0.12);
border-color: #0d6efd;
}
Key Features: repeat(auto-fit, minmax(280px, 1fr)) generates automatic responsive columns without media queries; transform: translateY(-6px) provides smooth GPU-accelerated micro-interactions.

>> 2.6 University Review & Examination Questions

* Q1: Contrast the CSS Box Model under 'box-sizing: content-box' versus 'box-sizing: border-box' with a numerical dimension calculation.
Answer: Under 'content-box' (CSS default), declared width and height apply strictly to the content box. Total rendered width is calculated as: Width_total = width + padding_left + padding_right + border_left + border_right. For an element with width: 300px, padding: 20px, and border: 5px, Total Width = 300 + 20 + 20 + 5 + 5 = 350px. If placed inside a 300px parent column, it overflows and breaks the layout. Under 'box-sizing: border-box', the declared width includes content, padding, and border. The browser automatically compresses content width: Content Width = 300 - (40 + 10) = 250px. Total rendered width remains exactly 300px, guaranteeing layout stability.
* Q2: When should a web developer select CSS Flexbox versus CSS Grid? Detail their architectural differences.
Answer: The fundamental distinction is dimensional scope: (1) Flexbox is One-Dimensional: It manages layout along a single axis at a time (either row OR column). It is content-driven (items determine their own width/flex distribution) and is optimal for interface components such as navigation menus, button groups, form input rows, and centering elements. (2) CSS Grid is Two-Dimensional: It manages rows AND columns simultaneously. It is layout-driven (the container establishes rigid structural tracks into which elements are placed) and is optimal for overarching page architectures, complex dashboards, multi-column image galleries, and overlapping component grids.

>> 2.7 Key Takeaways and Revision Points

* Specificity formula: Inline (1000) > ID (100) > Class (10) > Element (1).
* Always set box-sizing: border-box universally across all elements.
* Flexbox manages 1D alignments (main vs cross axis); CSS Grid controls 2D rows and columns.
* repeat(auto-fit, minmax(min, 1fr)) creates responsive grids without media queries.